Architecture.md v1.1

Chapter 14 — Development Contracts & Coding Standards

This chapter defines the mandatory development rules for AI OS. Every human developer and AI development agent (Gemini, Claude, Manus, or future AI assistants) must follow these standards.

These rules are mandatory unless superseded by a newer version of this architecture.

14.1 Purpose

The Development Contract ensures:

- Consistent code quality.
- Predictable architecture.
- Easier reviews.
- Easier maintenance.
- Reliable collaboration between AI systems and humans.

14.2 Single Source of Truth

Architecture.md is the technical authority.

If implementation conflicts with Architecture.md:

Implementation must be updated.

Architecture is never silently ignored.

14.3 Coding Principles

Every module must satisfy:

- Readability
- Simplicity
- Modularity
- Reusability
- Testability
- Maintainability

Shorter code is **not** automatically better code.

14.4 Naming Standards

Directories:

```
snake_case
```

Files:

```
snake_case.py
```

Classes:

```
PascalCase
```

Functions:

```
snake_case()
```

Variables:

```
snake_case
```

Constants:

```
UPPER_SNAKE_CASE
```

14.5 File Size Guidelines

Recommended maximum sizes:

- Module: ~500 lines
- Class: ~300 lines
- Function: ~50 lines

Large modules should be refactored into smaller components.

These are guidelines, not hard limits.

14.6 Documentation Requirements

Every public module must include:

- Purpose
- Responsibilities
- Dependencies
- Inputs
- Outputs

Every public function must include:

- Description
- Parameters
- Return value
- Possible exceptions

14.7 Error Handling Contract

Every recoverable error must return a structured response.

Responses include:

- Error Code
- Description
- Suggested Action
- Request ID

Unhandled exceptions are prohibited.

14.8 Logging Requirements

Every important operation logs:

- Start
- Completion
- Failure
- Duration

Sensitive information must never appear in logs.

14.9 Memory Rules

Worker Agents:

- May use temporary in-memory variables.
- Must never maintain persistent internal memory.
- Must store persistent data only through the Memory Service.

14.10 Communication Rules

Every module communicates only through approved architecture layers.

Forbidden:

- Direct Worker Agent communication.
- Direct database access.
- Hidden APIs.
- Global mutable state.

14.11 Dependency Rules

Allowed dependencies:

- Standard Library
- Approved third-party libraries
- AI OS Core Services

Avoid unnecessary external packages.

Every dependency should have a documented purpose.

14.12 Testing Standards

Every module should include:

- Unit Tests
- Integration Tests (where applicable)

Critical components additionally require:

- Failure Tests
- Permission Tests
- Boundary Tests

No feature is considered complete without tests.

14.13 Code Review Standards

Every implementation should be checked for:

- Architecture compliance
- Readability
- Performance
- Error handling
- Security
- Test coverage

The reviewer must not redesign the architecture during review.

Architectural concerns should be documented separately.

14.14 AI Collaboration Rules

ChatGPT

Responsible for:

- Architecture

- Specifications
- Technical decisions

Must not be overridden by implementation.

Gemini

Responsible for:

- Feature implementation
- APIs
- Backend logic
- Frontend implementation
- Tests

Must not redesign the architecture.

Claude

Responsible for:

- Code review
- Refactoring suggestions
- Performance analysis
- Documentation improvements

Must not rewrite architectural decisions.

Manus

Responsible for:

- Project scaffolding
- Templates
- Development automation
- Workflow generation

Must follow the approved architecture.

14.15 Version Control Rules

Every significant change should include:

- Version increment
- Change summary
- Author
- Date
- Reason

Architecture changes require explicit approval before implementation.

14.16 Performance Guidelines

Developers should prioritize:

- Correctness
- Readability
- Reliability

Performance optimization should occur only after correctness is established, unless performance